

Exact Posteriors for Values, Exact Costs for Choices: Two-Stage **Learned Branching** for Binary Linear Systems

고광종

고려대학교 산업경영공학과

- 요약

- **Learning-based 2-Stage Branching Guidance DFS 프레임워크 개발**

- 열거된 해와 subtree cost를 학습하여 DFS의 branching guidance를 결정하는 2-stage 프레임워크 개발
 - Stage 1: Supervised learning-based value (0 or 1) 결정
 - Stage 2: Reinforcement learning-based decision variable 결정
- 어느 값 / 어느 변수를 결정하는 두 가지 의사결정변수에 대해서 각각 교환 불가능한 학습 신호 有 확인

- 주요 contribution

- 실제 학습가능한 2가지의 유의미한 정보가 searching tree 내부에 존재했고, 이 2가지 정보는 교환 불가능함을 논리적/실험적으로 입증
- 제안하는 branching guidance DFS가 기존 LP-probing DFS보다 더 빠르고 정확하게 해를 탐색함을 입증
- 큰 사이즈의 instance에서 stage 1의 계산시간 이득이 잠식되나, stage 2의 RL이 다시 되살리는 것을 확인 → RL의 필수성
- 작은 사이즈의 instance를 학습해서 더 큰 사이즈의 instance로 적용하는 상향 전이가 유의미함을 실험적으로 입증
- In-tree 학습 접근 및 제안 네트워크의 GAT 구조가 일반적인 접근 대비 효과적이었는지 실험적으로 입증

- 한계

- 완결 Solver(CP-SAT, SCIP)에 비해서 9~15배 느림. But, 부족한 것은 branching 품질이 아닌 node 당 비용 (구현 자체의 효율성 문제)
- Supervised learning을 위해서 해당 사이즈 인스턴스의 모든 해 열거 필요

01 타겟 문제 분석 - 이진연립방정식 (BLS)

- 목표: 이진연립방정식 (BLS) 을 효율적으로 푸는 AI 기반 솔버 개발
 - 다차원 X선 투과정보를 완벽하게 만족하는 이미지 복원 문제 (산업계 비파과검사) → 이진연립방정식
 - 아래와 같이 기본적인 모델링이 가능하며 모든 제약을 만족하는 feasible solution을 찾는 문제 ($\approx$ 이진연립방정식의 해 찾기)
 - $A \in \{0, 1\}^{m \times n}$
 - $b \in \mathbb{Z}_{\geq 0}^m$

$$\begin{array}{ll} \textit{Find} & x \\ \textit{subject to} & Ax = b \\ & x \in \{0, 1\}^n \end{array}$$

- 실체가 존재하는 물체를 스캔했으므로, 해는 반드시 존재하나 다수의 해가 존재할 수 있음
- 실제 공정 라인에 원활히 적용하기 위해서는 **실시간에 가까운 고속 처리가 요구됨**
- 상기 문제는 **목적함수가 없어 bound 기반 pruning이 약하고, LP 완화 정보가 희소함**

01 타겟 문제 분석 – 이진연립방정식 (BLS)

- BLS를 기존의 수리적 최적화(ex. branch-and-cut)로 접근하기 어려운 이유
 - 목적함수가 존재하지 않음
 - 목적함수의 미존재로 bound 기반 pruning이 약함
 - Infeasible에 대한 판단만 유의미하게 가능하나, 그것도 아래의 이유로 무의미해짐
 - LP relaxation이 항상 실현 가능하나 정보가 희소함
 - 변수가 n 개, 제약이 m 개인데, $n > m$ 이라서 해공간은 거대한 차원의 아핀 큐브 공간과 하이퍼 큐브의 교집합이 형성
 - 이 polytope은 부피가 크고 vertex가 수 없이 많으며, 대부분이 실수임
 - LP relaxation이 제공하는 정보는 root에서 rounding 정확도 60.9%, random 보다 약간 나옴
 - Rounding이 원리적으로 불가능
 - 등식 제약으로만 구성되어 한 변수만 틀려도 행의 합이 깨어짐
 - 본 연구에서 자체적으로 측정한 LP rounding의 변수별 정확도는 root에서 60.9%로 random보다 약간 나은 수준임
 - Root propagation의 무력성
 - CP solver의 unit propagation은 “이 행에서 15개가 1이면 나머지는 전부 0”과 같은 규칙인데, 아무것도 고정되지 않은 root에서는 발동이 안됨
 - 탐색 Tree의 depth 5~8에서 조차 propagation은 논리적으로 결정된 변수의 1.2 ~ 5.3%만 탐지하는 것으로 확인됨 (10x25 in-tree 기준)

01 타겟 문제 분석 - 이진연립방정식 (BLS)

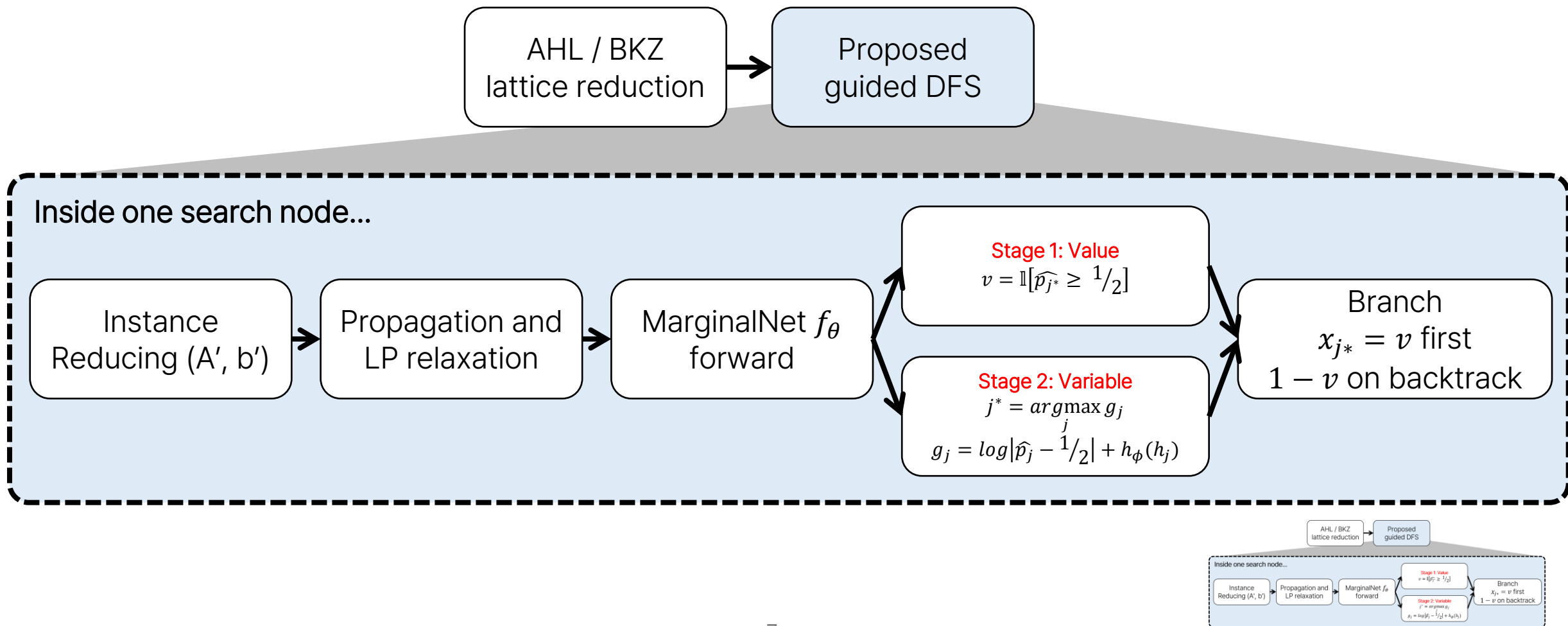
- BLS를 접근하는 우리의 논리
 - 결과적으로 Solver가 할 수 있는 일이 “어느 변수부터 시도할지 잘 고르는 것” → Branching heuristic을 학습 대상을 삼은 이유
 - 문제 자체가 Branching으로 환원되는 계열이므로 학습의 기여를 순수히 측정 가능함
 - Proposition 1. BLS에서 Conditioning = Reduction 임.
 - 어떠한 BLS 문제의 변수 집합 F 를 값 v 로 고정한 node는, 남은 변수 K 에 대한 더 작은 BLS $A_K y = b - A_F v$ 와 정확히 같다. 따라서 원래 문제의 conditional marginal (주변확률; 해들 중에서 이 변수가 1인 비율)은 reduction된 문제의 marginal이다.
- 기존의 학습 방법과 달라진 점
 - Marginal은 변수를 두 부류로 분할
 - FORCED: 정답 방향과 반대로 분기하면 subtree가 빔 → Backtrack 비용 확정 발생
 - OPEN: 어느 쪽으로 분기해도 해에 도달 → 비용 無
 - 오직 FORCED 오류만 비용 발생하고 이는 tree depth가 깊어질수록 폭증함 (10x25 in-tree 기준)
 - | Depth | 0 (root) | ... | 5 | 6 | 7 | 8 |
|-----------|----------|-----|-------|-------|-------|-------|
| FORCED 비중 | 6.3% | ... | 61.9% | 74.2% | 79.9% | 87.9% |
- 즉 root에서는 대부분의 branching 결정이 무의미하고, depth 8에서는 거의 모든 결정이 틀리면 비용 발생 → 기존 접근의 문제
 - In-tree 학습이 필수적으로 필요 → Branching Guidance

02 선행 연구

- 작성중

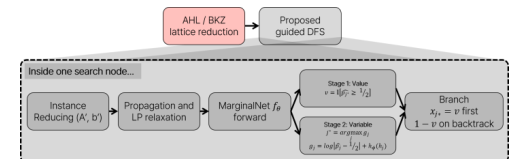
03 제안 프레임워크

- 제안 프레임워크 모식도



03 제안 프레임워크 - (1) AHL/BKZ Lattice Reduction

- AHL / BKZ Lattice Reduction
 - AHL (Aardal-Hurkens-Lenstra) 임베딩을 이용, $A \cdot x = b$ 의 해를 길이²이 $n+1$ 벡터로 번역
 - Lattice reduction 기법인 LLL 및 BKZ로 **짧은 벡터**를 찾아서 해독하여 검증
 - 매우 빠르고 (1998년부터 market-split의 기준 방법) 검증 절차가 있기 때문에 오답이 없음. But, 크기가 커질수록 거의 풀지 못함
- AHL 임베딩
 - Lattice는 기저 벡터들의 정수 계수 조합 전체를 의미
 - $\mathcal{L} =, c_0 \mathbf{m}_0 + c_1 \mathbf{m}_1 + \dots + c_k \mathbf{m}_k ; ; c_j \in \mathbb{Z},$
 - 같은 Lattice를 만드는 기저는 무수히 많지만, Lattice reduction은 그 중 짧고 서로 거의 직교하는 기저를 찾는 알고리즘
 - 축약된 기저의 벡터들은 lattice 안에서 가장 짧은 벡터들에 가깝다
 - 즉, 내가 찾는 해를 lattice의 아주 짧은 벡터가 되도록 lattice를 설계 → Lattice reduction을 통해 해를 찾음



03 제안 프레임워크 - (1) AHL/BKZ Lattice Reduction

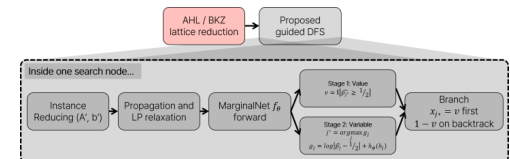
- AHL 임베딩 Example

- 예시 문제:

- $x_1 + x_2 + x_3 = 2$
- $x_2 + x_3 + x_4 = 1$
- Solution: $S = \{(1, 0, 1, 0), (1, 1, 0, 0)\}$

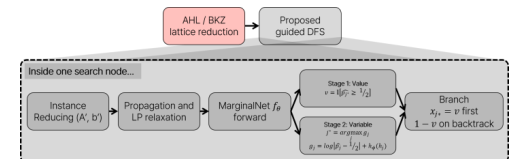
- 임베딩: 원 문제를 5개의 벡터로 치환

- 7칸짜리 벡터 5개 생성. 7칸 = 변수부 4칸 + 표시부 1칸 + 제약부 2칸. $N=100$.
- 변수 x_j 마다 벡터 하나씩 생성. 규칙: 변수부에는 j 번째 칸에 2, 표시칸에 0, 제약부에는 x_j 가 각 제약에 관여하면 $1 * N$
- 우변 b 를 위한 벡터 생성. 규칙: 변수부 전부 1, 표시칸 1, 제약부에는 $b * N$
 - $row_0 : [2, 0, 0, 0] \mid 0 \mid [100, 0] \leftarrow x_1$ 은 1행에만 등장 2행에는 없어서 $[1, 0]$
 - $row_1 : [0, 2, 0, 0] \mid 0 \mid [100, 100] \leftarrow x_2$ 는 모두 등장 $[1, 1]$
 - $row_2 : [0, 0, 2, 0] \mid 0 \mid [100, 100] \leftarrow x_3$ 도 모두 등장 $[1, 1]$
 - $row_3 : [0, 0, 0, 2] \mid 0 \mid [0, 100] \leftarrow x_4$ 는 2행에만 등장 $[0, 1]$
 - $row_4 : [1, 1, 1, 1] \mid 1 \mid [200, 100] \leftarrow b$ 는 $(2, 1)$
- 이 5개가 기저이고, Lattice $L = c_0 \cdot row_0 + c_1 \cdot row_1 + c_2 \cdot row_2 + c_3 \cdot row_3 + c_4 \cdot row_4$



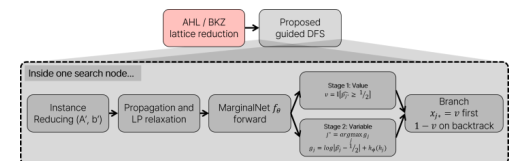
03 제안 프레임워크 - (1) AHL/BKZ Lattice Reduction

- Lattice에서 “**짧은 벡터**”를 찾는 것의 의미
 - 실제 해 $(1, 0, 1, 0)$ 을 $c_0 \sim c_3$ 에 대입하고, c_4 에는 -1 대입하는 경우...
 - $+1 \cdot row_0 : [2, 0, 0, 0] \mid 0 \mid [100, 0]$
 - ~~$+1 \cdot row_1 : [0, 2, 0, 0] \mid 0 \mid [100, 100]$~~
 - $+1 \cdot row_2 : [0, 0, 2, 0] \mid 0 \mid [100, 100]$
 - ~~$+1 \cdot row_3 : [0, 0, 0, 2] \mid 0 \mid [0, 100]$~~
 - $-1 \cdot row_4 : [1, 1, 1, 1] \mid 1 \mid [200, 100]$
 - 합 = $[1, -1, 1, -1, -1, 0, 0]$ 이고, 길이²은 $1+1+1+1+1=5$ 임
 - 모든 정답은 $[\pm 1, \pm 1, \pm 1, \pm 1, -1, 0, 0]$ 꼴이고, 길이는 일반적으로 $n+1$ 인 짧은 벡터임
 - 만약 정답이 아닌 경우 제약부에 $[100, 100]$ 부분이 남아서 길이가 폭발적으로 증가함- Lattice Reduction: Lattice에서 짧은 벡터를 찾아내는 것 (LLL & BKZ)
 - $n+1$ 개의 기저벡터를 가지고 한 벡터에서 다른 벡터의 정수배를 빼서 더 짧아지면 바꾸는 과정을 반복
 - 반복이 끝나면 기저는 짧은 벡터들로 바뀌고, 그 안에서 길이²이 $n+1$ 인 정답벡터를 도출 → 검산 검증까지 하는 건전한 방법
 - LLL과 BKZ는 한번에 몇 개의 벡터를 같이 보지만 다른 Lattice reduction 알고리즘
 - 열 순열 10개를 시도하고 각 시도마다 LLL 1회 → BKZ 수행 (LLL이 먼저 큰 성분을 소거하면 효율적)



03 제안 프레임워크 - (1) AHL/BKZ Lattice Reduction

- AHL/Lattice Reduction이 큰 문제에서는 실패하는 이유?
 - 21x60과 같이 문제 사이즈가 큰 경우 벡터가 61개, 칸이 82개인 Lattice가 생성됨
 - 이 격자에는 kernel 벡터 (표시칸 0, 변수부 짝수)가 39개나 있고, 그중 짧은 벡터의 길이²은 40~52로 정답보다 짧음
 - kernel 벡터는 $A \cdot z = 0$ 인 정수 벡터로 A를 0 방향으로 보내버리는 영공간 벡터로 실수 공간 해에 해당함
 - Lattice reduction 알고리즘의 종류인 LLL과 BKZ는 짧은 벡터로 기저를 채우기 때문에 기저 61자리가 kernel 벡터와 긴 벡터로 채워지고, 정답 벡터는 잘 도출되지 않음
- 프레임워크의 전단계에 포함된 이유?
 - 1998년 이후 지금까지도 Market-split 문제에서의 기준 방법
 - market-split 문제는 우리의 문제와 유사하게 LP 완화가 무정보하다는 특징이 있으며, Aardal–Hurkens–Lenstra(1998)에서 Lattice reduction으로 해당 문제를 해결한 격자 기반 접근이 기록 보유, Wassermann(2025)까지 이어짐
 - 풀 수만 있다면 훨씬 저렴함
 - 21x60 정도의 사이즈만 되어도 성공하면 제안 대비 100배 싸고, 실패해도 손실이 전체 시간의 1% 정도 수준임
- “Classic SOTA가 실패하는 곳에서 제안하는 학습 방법이 무엇을 더하는가?”를 실질적으로 보여줌



03 제안 프레임워크 - (2) Guided DFS

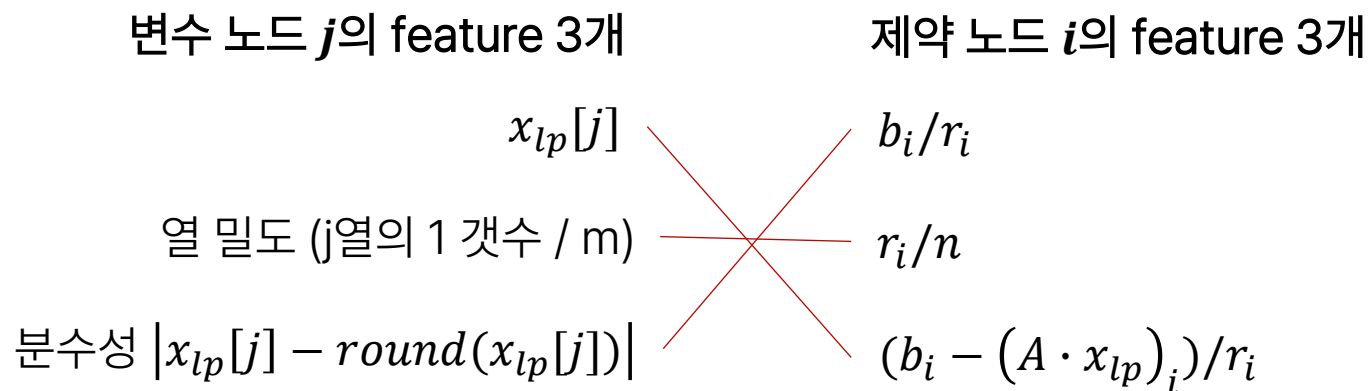
- Guided DFS
 - 앞선 단계에서 모두 실패하면 모든 정보를 폐기하고 Guided DFS를 수행
 - 한 노드=부분할당 에서 발생하는 알고리즘 순서
 - (1) Instance reducing
 - (2) Propagation and LP relaxation
 - (3) MarginalNet forward
 - (4) Value / Variable 결정
- Instance reducing
 - 각 노드마다 원래 문제가 아닌 고정된 변수를 소거한 축소 인스턴스를 부여받음
 - $x_j = v$ 로 고정한다면, A 에서 j 열을 제거하고 b 에서는 고정된 기여분을 차감 (Proposition 1)
 - 작은 사이즈 인스턴스 학습 때 노드와 큰 사이즈 인스턴스 추론 시 트리 깊은 곳에서 만나는 노드가 같은 형태 → 크기 전이 근거
 - 노드 진입 시 3가지 체크:
 - bound check: 어떤 행이 $b_i < 0$ 이거나 $b_i >$ 그 행의 남은 변수의 갯수이면 infeasible로 노드를 버림
 - 빈 행 제거: 변수가 다 빠져나가서 A 행합이 0이면 행 삭제. 행이 하나도 안남으면 모든 제약 만족 → 해 발견
 - 남은 변수가 없으면 $A \cdot x = b$ 계산 후 해 반환

03 제안 프레임워크 - (2) Guided DFS

- Propagation and LP relaxation
 - 각 행 i 에 대해 남은 우변 b_i 와 남은 변수의 수 r_i 를 확인한 후:
 - $b_i = 0 \rightarrow$ 그 행의 미할당 변수 전부 0으로 고정
 - $b_i = r_i \rightarrow$ 그 행의 미할당 변수 전부 1로 고정
 - $b_i < 0$ or $b_i > r_i \rightarrow$ Infeasible
 - 같은 변수가 0과 1로 동시에 강제 $\rightarrow$ Infeasible
 - 고정이 한 번이라도 발생하면 또 다른 행을 Propagation할 수 있으니 변화가 없을 때까지 반복
 - 목적함수는 똑같이 0으로 LP relaxation 동작
 - Pruning: LP가 Infeasible이면 정수 해도 없으므로 노드 버림
 - Feature generation: LP해를 생성하여 다음 단계인 MarginalNet의 입력으로 사용
 - 이때 LP는 warm-start를 사용
 - 노드마다 LP를 새로 만들지 않고, 원래 n 개 변수위에 LP 하나를 탐색 내내 유지
 - 노드는 변수 j 의 하한 = 상한 = v 라는 bound 변경 집합으로만 표현되고 부모-형제 노드와는 bound 몇 개만 다르므로 dual simplex가 이번 basis에서 몇 번의 pivot만으로 종료

03 제안 프레임워크 - (2) Guided DFS

- MarginalNet forward
 - 입력: 축소된 인스턴스 (A, b)와 x_{lp} 로 만든 bipartite graph



- GAT 4층(Graph Attention; Hidden 64, 총 21,761 파라미터)이 변수 $\rightarrow$ 제약, 제약 $\rightarrow$ 변수 방향으로 메시지를 번갈아 주고받아 변수 임베딩 h_j 생성 $\rightarrow$ 해당 임베딩 위에서 head 3개
 - marginal head: $m_j = \text{Linear}(h_j)$, $\hat{p}_j = \text{sigmoid}(m_j)$ $\leftarrow$ Stage 1에서 학습
 - policy head: $g_j = \log|\hat{p}_j - 0.5| + h_\phi(h_j)$ $\leftarrow$ Stage 2에서 학습
 - value head: $V = \text{MLP}([\text{mean } h, \max h, \text{mean } h_c, \max h_c, \log n, \log m])$ $\leftarrow$ 학습 전용 Critic

03 제안 프레임워크 - (2) Guided DFS

- Stage 1 → value (값 결정)
 - $\hat{p}_j$ 는 “이 축소 인스턴스의 해들 중 $x_j=1$ 인 비율 (Conditional marginal)”의 예측치
 - 학습 라벨은 10x25 트리 (깊이 0~12)에서 전수열거로 계산한 정확한 marginal. 손실함수는 Binary Cross Entropy
 - $v = 1 \quad \text{if} \quad \hat{p}_j^* \geq 0.5 \quad \text{else} \quad 0$
- Stage 2 → variable (변수 선택)
 - Marginal head는 그대로 두고 Policy head h_ϕ 만 학습. 값 결정은 그대로
 - $J^* = \operatorname{argmax}_j g_j = \operatorname{argmax}_j [\log|\hat{p}_j - 0.5| + h_\phi(h_j)]$
 - h_ϕ 마지막 층이 0으로 초기화되어 있으므로 학습 전에는 $\log|\hat{p}_j - 0.5|$ 만 작동
 - 학습은 h_ϕ 가 이 순서를 얼마나 고쳐야 하는지를 배우는 과정
 - 노드 u 에서 변수 j 를 고르고 그 subtree를 다 파는데 든 노드의 수 $c(u)$ 를 DFS 자체 기록에서 획득 → 보상 = $-\log c(u)$
 - critic V가 예측한 기대비용과의 차이를 advantage로 써서 “예상보다 싸게 끝난 선택”의 logit을 향상
- 이러한 과정을 반복하여 DFS 탐색 수행

- 본 제안 프레임워크를 검증하기 위해서 하기의 4가지를 질문으로 제시
 - 기존의 탐색 방법보다 성능이 좋은가?
 - 더 큰 사이즈의 문제 인스턴스로 상향 전이 적용이 가능한가?
 - 완결 Solver를 대체할 수 있는 수준인가?
 - 다양한 관점에서의 Ablation Study에서도 본 제안 프레임워크의 모든 요소(stage1+stage2)가 필수적인가?
- 실험 세팅
 - 컴퓨팅 환경: Intel(R) Core Ultra 7 265KF (3.9 GHz) / 64GB RAM / RTX 5080 (비교 실험 시 GPU 미사용)
 - 공통 루프: node 당 같은 warm-start relaxation, 동일 propagation, 동일 값 규칙, 동일 backtrack
 - 인스턴스 당 컴퓨팅 시간 제한: 25: 60초 / 50: 300초 / 60: 600초 / 70 및 80: 1200초
 - 병렬성: 20코어 머신에서 동시성 6. 모든 계산은 단일 스레드
- 문제 인스턴스 사이즈
 - 10x25, 18x50, 21x60, 24x70, 28x80

04 실험

- 비교 arms 정의

Arm	변수 선택	첫 값	네트워크
Random	무작위	무작위	없음
LP	LP 해가 가장 정수에 가까운 변수	$\text{round}(x_{lp_i})$	없음
Stage 1	$\text{argmax} \hat{p}_j - 0.5 $	$\mathbb{I}[\hat{p}_j \geq 0.5]$	MarginalNet w/o policy head
Stage 1+2	$\text{argmax} g_j$	$\mathbb{I}[\hat{p}_j \geq 0.5]$	MarginalNet w/ policy head

- Random은 이 인스턴스가 정말 어려운가에 대한 검사용
- LP는 일반적인 DFS에 가장 가까우며 MILP 솔버의 고전적 규칙 → 이 비교에서 나온 차이는 순수하게 branching 결정의 질
 - 노드 진입 시 bound check, 빈행 제거, propagation, warm-start LP, 시간 예산은 모두 동일하며 guide 함수만 다름
- Market-split-style 구조의 어려운 난이도의 인스턴스 생성 (Cornuéjols–Dawande(1998))
 - 각 행이 변수의 절반쯤을 포함하는 밀집 시스템 (기존 생성 인스턴스)
 - $n/2$ 개를 1로 심은 해 + vertex spread 적용

04 실험

- 기존 탐색 방법보다 성능이 좋은가?

- A. 그렇다. 탐색 트리 크기는 제안 방법이 항상 더 좋고, 계산시간은 트리가 forward 비용을 상각할 만큼 커진 18x50부터 좋아진다.

인스턴스 크기 (시간 제한)	전략	해결	탐색 트리 크기 중앙값	계산시간 중앙값	노드처리량/s
10x25 (60s)	LP	30/30	46	0.03s	1,767
	Stage 1	30/30	24	0.07s	376
18x50 (300s)	LP	30/30	4,224	2.43s	2,066
	Stage 1	30/30	1,085	1.82s	731
	Stage 1 + 2	30/30	424	0.52s	820
21x60 (600s)	LP	100/100	79,609	41.2s	1,966
	Stage 1	100/100	13,386	16.5s	796
	Stage 1 + 2 (Seed1)	100/100	4,940	6.8s	730

※ Lattice OFF

- 노드 감소 비율이 크기와 함께 증가
- 10x25에서는 Stage 1이 LP보다 느림 → 24노드 트리 < marginalNet forward 비용으로 사료
- Random branching은 18x50 이상에서 한 인스턴스도 풀지 못함 → 이 문제가 어렵다는 근거

04 실험

- 더 큰 사이즈의 문제 인스턴스로 상향 전이 적용이 가능한가?
 - A. 그렇다. 더 작은/큰 사이즈로의 양방향 전이 모두 가능했다.

학습 → 평가	비교대상	탐색 트리 크기 비	계산시간 비
Stage 1: 25 → 50	LP	3.9x	1.3-1.4x
Stage 1: 25 → 60	LP	5.5x	2.0-2.3x
Stage 1: 25 → 70	LP	3.8x	0.7-1.1x
Stage 1+2: 60 → 50	Stage 1	2.6x	1.8-2.2x
Stage 1+2: 50 → 60	Stage 1	2.0x	1.7-1.9x
Stage 1+2: 60 → 70	Stage 1	2.7x	2.3-3.3x
Stage 1+2: 50 → 70	Stage 1	2.2x	2.1-2.7x

- Stage 1은 10x25에서만 학습하고 25 → 50/60/70 노드 우위 유지
- Stage 2는 50<->60 , 60→70, 50→70 까지 상향 전이 우위 유지
- Stage 1만으로는 시간 우위가 70 상향 전이 부근에서 사라지지만, Stage 2가 그 우위를 되살려 더 위로 가져감

04 실험

- 완결 Solver를 대체할 수 있는 수준인가?
 - A. 아니다. CP-SAT와 SCIP MILP가 훨씬 앞선다. 그러나 원인은 branching 품질이 아닌 노드당 처리량에서 벌어졌다.

계산시간 중앙값

인스턴스 크기	CP-SAT	SCIP	Stage 1	Stage 1+2
10x25	0.002s	0.009s	0.07s	-
18x50	0.046s	0.385s	1.82s	0.52s
21x60	0.79s	2.59s	16.5s	6.8s
24x70	16.1s	11.7s	358-457s	125-185s

	CP-SAT	Stage 1+2
Branch/노드 수 (중앙값)	18,827	4,040-8,873
노드 처리량/s	19,782	~700

- 제안 방법은 해를 찾을 때까지 CP-SAT보다 2-4배 적게 탐색 트리를 탐색. But, 노드 처리량은 CP-SAT가 28배 빠름
 - branch와 node는 같은 단위가 아니므로 CP-SAT가 우리보다 적게 보는 것은 아님
 - branch 카운터는 restart 포함, clause learning 건너뛰기 미포함
- CP-SAT는 C++기반의 엔진, 제안 방법은 Python 기반 구현으로 구현 자체의 효율성 차이가 있음

04 실험

- 다양한 관점에서의 Ablation Study에서도 본 제안 프레임워크의 모든 요소가 필수적인가?
 - A1. Stage 1의 학습신호와 Stage 2의 학습신호는 불교환성을 갖는다. 값 정확도를 올린다고 해서 탐색 트리의 크기가 줄어들지 않고, 비용을 줄인 정책은 값 확신이 낮은 변수를 선택한다. 즉, 두 신호는 서로 대체 불가하다.

방법	변경점	값 정확도 (rounding 대비)	탐색 트리 크기 중앙값
Stage 1	-	+ 5.4 / 9.7	13,386
Stage 1 변형	테스트 인스턴스 사이즈로 학습 풀의 in-tree 상태 학습	+ 6.1 / 10.7	21,337

측정	Stage 1	Stage 1+2
선택 변수의 $ \hat{p} - 1/2 $	0.43	0.26
선택 변수의 확신 순위	1위 (정의상)	중앙값 15위
선택 변수가 FORCED이고 정확할 확률	82.5%	57.5%
Stage 1과 다른 변수를 선택한 비율	-	95%
21x60 100개 노드에서의 변수 선택 후 탐색 트리 크기 (중앙값)	13,386	4,940 (Seed 1)

- (표 1) 학습 데이터 변경을 통해서 값 정확도라는 지표를 향상시켰는데 탐색 트리 크기는 증가함
- (표 2) Stage 1 기준에서는 Stage 2의 선택이 모든 면에서 나쁘지만 탐색 트리 크기는 절반 이하임

04 실험

- 다양한 관점에서의 Ablation Study에서도 본 제안 프레임워크의 모든 요소가 필수적인가?
 - A2. Stage 1에 비해서 Stage 1+2가 유의미해지는 구간이 인스턴스 사이즈가 커질수록 분명하게 존재한다.

조건	탐색 트리 크기 비 (Stage 1 대비)	계산 시간 비 (Stage 1 대비)
18x50, 같은 크기에서 학습	0.9x	1.1-1.4x
21x60 → 18x50	2.6x	1.8-2.2x
21x60	2.3-2.4x	2.0-2.4x
24x70	2.7x	2.3-3.3x
28x80	해결 인스턴스 수 8/30 → 18/30	

- Stage 1+2는 트리가 클 때에만 의미가 있고, 클수록 그 의미가 커진다. 작은 트리에서는 Stage 1으로 충분

04 실험

- 다양한 관점에서의 Ablation Study에서도 본 제안 프레임워크의 모든 요소가 필수적인가?
 - A3. AHL Lattice Reduction에서 Learning이 보완할 부분이 존재한다.

크기	seed 0	seed 1	seed 2	해결율
10x25	30/30	30/30	30/30	100%
18x50	27/30	28/30	24/30	80-93%
21x60	16/30	14/30	10/30	33-53%
24x70	3/30	4/30	2/30	7-13%
28x80	0/30	1/30	0/30	0-3%



Arm	해결	해결율	계산 시간 중앙값
lattice 단독	2-4/30	7-13%	0.3s
LP	23-25/30	77-83%	537-644s
Stage 1	29/30	97%	358-457s
Stage 1+2 60에서 학습	30/30	100%	125-185s
Stage 1+2 50에서 학습	30/30	100%	135-165s

Lattice OFF

- Lattice reduction 계열 방법은 계산시간이 항상 매우 빠르나 사이즈가 커질 수록 해결율 급락
- But, 제안 방법을 통해서 Lattice reduction이 저비용으로 해결하지 못한 인스턴스를 처리할 수 있음을 24x70 사이즈에서 확인

- 다양한 관점에서의 Ablation Study에서도 본 제안 프레임워크의 모든 요소가 필수적인가?
 - A4. In-tree 학습 접근은 유효했으며, MarginalNet에서 차용한 GAT 구조는 일반적인 MLP 대비 우수한 성능을 보였다.

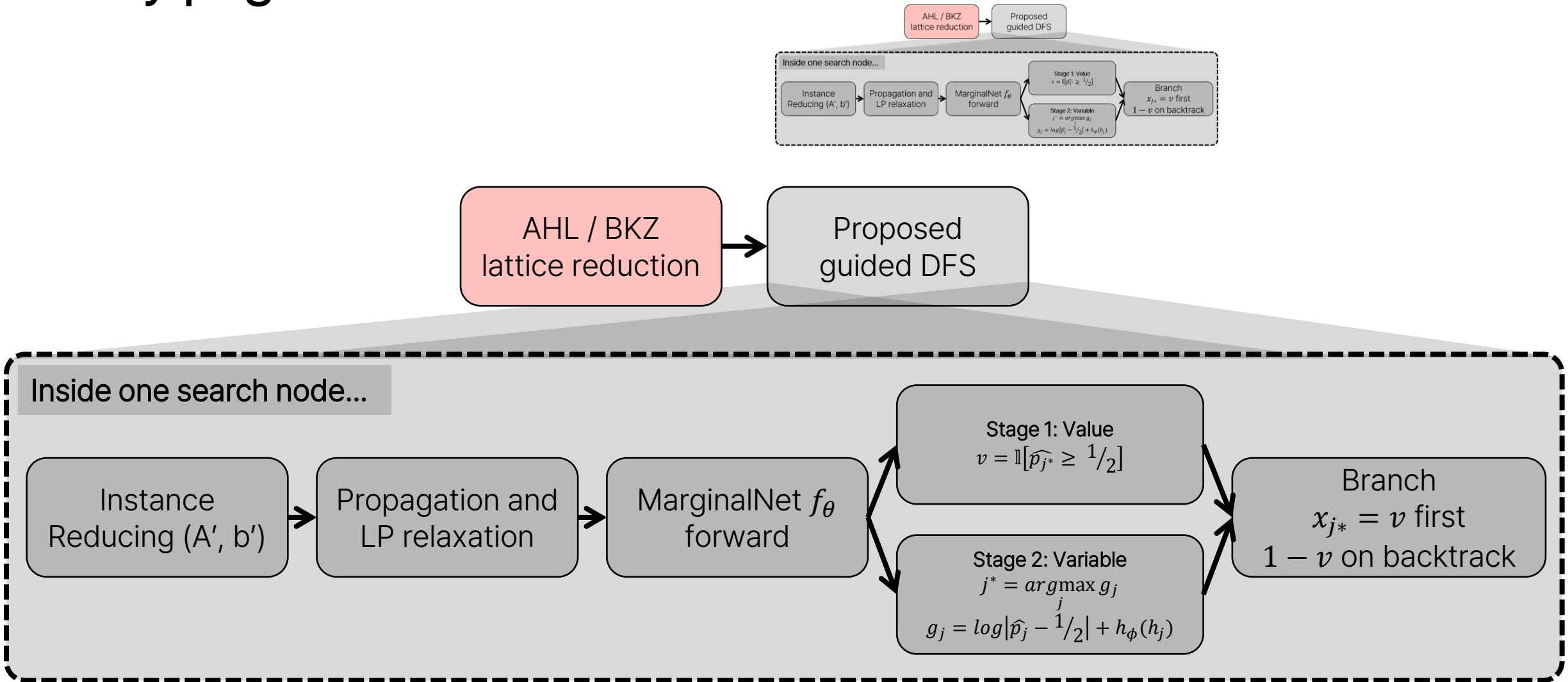
FORCED only 측정	정확도 (all depths)	정확도 (depth 8)
Bayes ceiling (상한)	100%	100%
(Proposed) GAT 구조 + in-tree 학습	93.2%	89.4%
MLP 구조 + in-tree 학습	89.5%	86.2%
GAT 구조 + root 통제 학습	88.0%	81.9%
LP rounding	87.9%	85.3%

- 10x25 in-tree 2,640 node 기준으로 분기 정확도 측정에서 제안하는 GAT 구조와 in-tree 학습이 가장 정확도가 높음
- in-tree 학습 포기 시 LP rounding보다도 낮은 성능
- 제안의 GAT 구조를 MLP로 변경 시, LP보다는 낮지만 제안보다는 낮은 성능

THANK YOU



Dummy pages



r_i : 제약 행 i 에 남아있는 변수 갯수

m_j : 변수 j 의 marginal logit

h_j : trunk 출력 (변수 j 의 임베딩, 입력 특징 3개가 GAT 4층을 거쳐 출력된 벡터)

h_ϕ : policy head라는 작은 MLP (64, 64, ReLU, 1). h_j 를 넣으면 실수 하나가 나옴